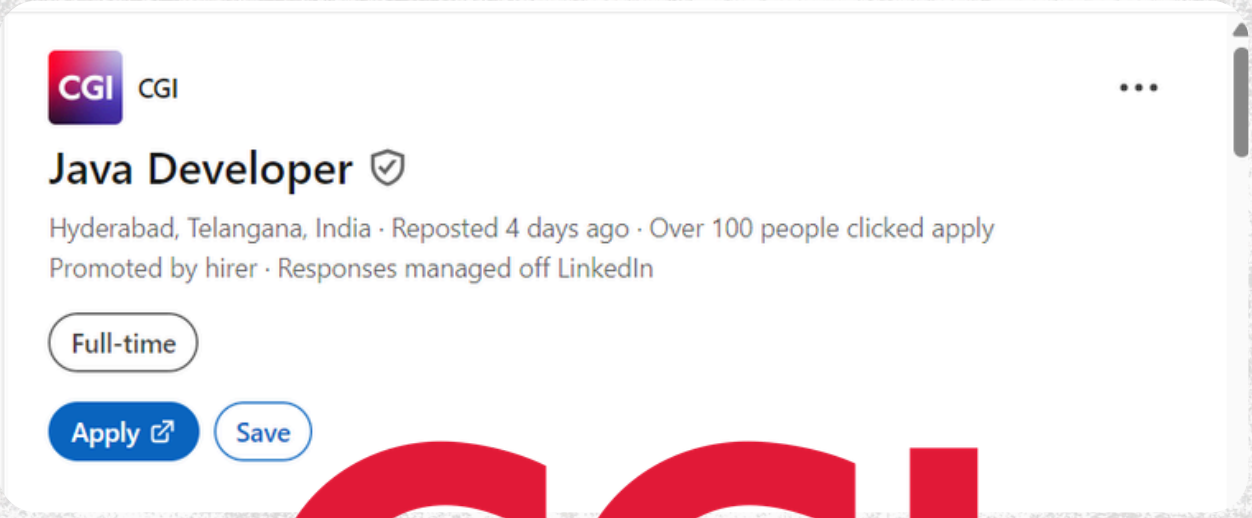
prominent Academy



CGI

Java Developer 1st Technical Round: Toughest Scenario-Based Interview Questions for 4-6YOE

www.prominentacademy.in

+91 98604 38743

Cloud-Native Java Application Design

1.
Your application was originally designed for a single server but now must run simultaneously across multiple AWS regions. What application-level changes would you make before deployment?
2.
A new feature requires processing user requests differently depending on the AWS region where they originate. How would you design the backend without duplicating business logic?
3.
Your backend currently assumes local file storage, but the application is moving to a cloud-native architecture. Which parts of the application would require redesign?
4.
A microservice generates temporary files during request processing. After migrating to containers, files occasionally disappear before processing completes. How would you redesign the workflow?
5.
Your application uses timestamps generated by different servers. Users report inconsistent ordering of events. How would you design around distributed clocks?
6.
How would you determine whether a backend service is truly stateless before deploying multiple instances?
7.
Your application generates PDF reports that take several minutes to create. How would you redesign the solution to keep APIs responsive?
8.
A cloud-native service must continue operating even while configuration updates are being rolled out. How would you design configuration refresh safely?
9.
How would you identify hidden assumptions in legacy Java code that prevent horizontal scaling?
10.
A customer requests tenant-specific business rules while keeping one shared application deployment. How would you organize the backend architecture?
11.
A business operation depends on five independent internal components. How would you minimize overall response time without changing business behavior?
12.
How would you determine whether a backend operation should remain request-driven or become scheduled processing?
13.
Your service processes customer uploads whose size varies from a few KB to several GB. How would you design for predictable resource utilization?
14.
A new compliance requirement mandates that business operations remain reproducible months later. How would you preserve execution context?
15.
How would you evaluate whether a cloud-native redesign actually improved maintainability instead of simply changing deployment infrastructure?

Spring Boot & REST Engineering

16.
Your API currently exposes implementation-specific field names that business teams want to change. How would you redesign responses while protecting existing clients?
17.
A REST endpoint supports dozens of optional filters, making controller logic increasingly complex. How would you simplify request processing?
18.
A business process requires validating information obtained from three different downstream services before accepting a request. How would you organize the validation flow?
19.
A client application occasionally disconnects before receiving a response, but the backend continues processing. How would you handle abandoned requests?
20.
How would you identify duplicate business validations implemented across multiple Spring Boot services?
21.
A REST API must support mobile clients with slow network connections. Which backend optimizations would you consider before changing the frontend?
22.
A response object contains fields that require expensive computation, but most clients never use them. How would you redesign the API?
23.
How would you detect unnecessary service-to-service REST calls during one customer request?
24.
A backend service receives requests from automated systems that occasionally resend incomplete payloads. How would you protect business consistency?
25.
How would you design REST APIs that remain understandable after years of feature additions?
26.
A downstream API introduces rate limits. How would your service adapt without affecting customer experience?
27.
A business workflow requires temporary request state spanning several API calls. How would you manage this without violating stateless service principles?
28.
How would you determine whether API composition belongs inside the backend or API Gateway?
29.
A controller method contains repeated authorization checks. How would you centralize the implementation?
30.
How would you verify REST contract compatibility before deploying a new service version?

AWS Services

31.
An EC2 Auto Scaling event launches additional instances, but startup time becomes the bottleneck. How would you reduce instance warm-up time?
32.
Your application stores millions of small files in S3. Over time, retrieval patterns change significantly. How would you reorganize storage strategy?
33.
An RDS maintenance window overlaps with critical business processing. How would you minimize customer impact?
34.
A Lambda function invokes multiple downstream services. One dependency occasionally responds slowly. How would you redesign execution flow?
35.
API Gateway receives traffic from both trusted internal systems and public consumers. How would you organize request handling policies?
36.
CloudWatch indicates healthy infrastructure, but customers still report slow transactions. Which application-level metrics would you introduce?
37.
An ECS deployment completes successfully, but newly launched tasks receive almost no traffic. Which deployment components would you verify?
38.
A service running on EC2 performs background processing that should continue even after application restarts. How would you redesign it?
39.
Your application stores customer-generated reports in S3. Business users need immediate search capability after upload. How would you design the workflow?
40.
How would you evaluate whether Lambda or Spring Boot is more appropriate for an operation executed thousands of times per minute?

AWS Services

41.
An RDS read workload grows much faster than write workload. What architectural changes would you evaluate?
42.
A CloudWatch alarm triggers repeatedly, but customer impact remains minimal. How would you improve alert quality?
43.
How would you verify that cloud resource scaling aligns with actual business demand rather than inefficient application behavior?
44.
An API Gateway deployment introduces unexpected request transformations. How would you isolate configuration changes?
45.
A service running inside ECS requires access to large reference datasets. How would you avoid repeatedly downloading them during task startup?
46.
How would you design S3 object organization to support efficient lifecycle management over several years?
47.
An AWS deployment spans multiple environments with different compliance requirements. How would you organize infrastructure configuration?
48.
How would you determine whether CloudWatch metrics accurately represent customer experience?
49.
Your backend depends on multiple AWS services that evolve independently. How would you reduce coupling between business logic and cloud APIs?
50.
How would you validate disaster recovery readiness before a production audit?

Distributed Systems & Scalability

51.
Customer demand grows unevenly throughout the day, causing one business capability to scale much faster than others. How would you isolate scaling requirements?
52.
A business operation requires combining data generated at different times by several services. How would you maintain consistency?
53.
How would you identify whether application bottlenecks originate from coordination rather than computation?
54.
A backend service experiences sudden bursts of short-lived requests. How would you smooth processing without affecting customer response times?
55.
A service becomes difficult to evolve because many downstream applications depend on it. How would you reduce dependency risk?
56.
How would you determine whether synchronous communication remains appropriate as business complexity increases?
57.
A business workflow includes optional processing steps enabled for only some customers. How would you design extensibility?
58.
How would you organize cross-service validation rules without creating circular dependencies?
59.
A backend operation succeeds technically but exceeds acceptable customer waiting time. How would you redesign the workflow?
60.
How would you evaluate whether application partitioning improved operational independence?
61.
How would you detect duplicated processing occurring across independent services?
62.
A shared business service becomes a deployment bottleneck for unrelated teams. How would you reorganize ownership?
63.
How would you verify scalability improvements before increasing production traffic?
64.
A backend platform gradually accumulates multiple communication patterns. How would you simplify integration architecture?
65.
How would you ensure new services follow established resilience practices without depending solely on documentation?

SQL, Git, Maven & Debugging

66.
A SQL query performs consistently during testing but unpredictably in production. Which environmental differences would you investigate first?
67.
How would you detect business calculations accidentally duplicated between Java code and SQL procedures?
68.
A Git merge introduces no compilation errors but changes runtime behavior. How would you investigate semantic conflicts?
69.
A Maven dependency upgrade unexpectedly changes application startup order. How would you determine the root cause?
70.
How would you organize Git history to simplify production issue investigations months later?
71.
A production defect appears only after multiple incremental deployments. How would you identify the responsible change efficiently?
72.
How would you validate SQL optimizations against realistic production workloads instead of synthetic benchmarks?
73.
How would you investigate intermittent failures occurring only after long application uptime?
74.
A build succeeds locally but packaged artifacts differ between developers. How would you enforce build consistency?
75.
How would you determine whether a debugging session is masking the original timing-related problem?
76.
A feature requires several database schema changes spread across multiple releases. How would you organize migrations safely?
77.
How would you verify that Git branching strategy supports parallel feature delivery without increasing production risk?
78.
A production issue leaves almost no useful logs. What additional diagnostic capabilities would you introduce?
79.
How would you detect unnecessary rebuilds increasing CI execution time?
80.
How would you compare alternative debugging strategies before modifying production code?

Agile, Delivery & Production Support

81.
A Product Owner changes acceptance criteria halfway through development. How would you evaluate technical impact before continuing implementation?
82.
How would you estimate work for a feature requiring investigation of unfamiliar AWS services?
83.
A sprint includes both architectural improvements and urgent customer fixes. How would you prioritize engineering effort?
84.
How would you communicate technical implementation risks to business stakeholders without overwhelming them with engineering details?
85.
A production deployment succeeds technically, but customer adoption remains lower than expected. Which engineering signals would you analyze?
86.
A recurring production issue has different symptoms every time it appears. How would you organize long-term investigation?
87.
How would you reduce onboarding time for developers joining a large Spring Boot and AWS project?
88.
A code review reveals no functional defects but identifies long-term maintainability concerns. How would you justify delaying the merge?
89.
How would you verify that sprint retrospectives lead to measurable engineering improvements?
90.
A production support issue interrupts sprint work almost every week. How would you improve delivery predictability?

Agile, Delivery & Production Support

91.
How would you evaluate whether technical documentation remains useful after multiple feature releases?
92.
A backend service receives increasing feature requests from multiple product teams. How would you prevent uncontrolled scope growth?
93.
How would you identify engineering work that provides the highest long-term operational value?
94.
A release passes all automated tests but fails during business validation. Which quality gates would you strengthen?
95.
How would you measure backend engineering success beyond feature completion?
96.
You join a CGI project that includes multiple Spring Boot microservices, AWS-native deployments, ECS workloads, CloudWatch monitoring, API Gateway integrations, RDS databases, and frequent customer enhancement requests. What would your first 30-day technical assessment plan include?
97.
If you were responsible for reviewing this project's architecture before onboarding new developers, which technical areas would you document first and why?
98.
A client asks how your backend platform will remain maintainable after five years of continuous feature additions. Which engineering principles would you highlight?
99.
Imagine you become the technical owner of CGI's cloud-native Java platform. How would you evaluate its scalability, operational readiness, deployment strategy, AWS utilization, security posture, and long-term maintainability before approving future development?

Prominent Academy me:

- ✔️ Real production scenarios
- ✔️ Pressure-based mock interviews
- ✔️ Decision-making framework
- ✔️ Interview Prep
- ✔️ Interview calls (Unlimited Till placement)
- ✔️ Pay After Placement option

WhatsApp

📞 +91 98604 38743